第2回：内積・外積のC#実装例

詳細解説 と 宿題の問題・解答 の共有数値例を、6題材・73件の自動チェックで確かめるコンソール教材。

`double` のタプルで数式を直接計算し、`System.Numerics` の `float` の結果と比較する。入力はソース内の固定値で、引数や標準入力は使わない。AutoCAD・GUI・追加NuGetパッケージは不要。

必要環境

- **.NET 10 SDK**（対象フレームワーク：`net10.0`）。ランタイムだけではビルドできない。
- Visual Studioでは **Visual Studio 2026（18.0以上、Windows）** と、Visual Studio Installerの「.NETデスクトップ開発」ワークロード・.NET 10 SDKを使う。SDKは使用するVisual Studioに対応したものを導入する。公式の導入・対応表 を参照。
- CLIでは.NET 10 SDKを導入したWindows・Linux・macOSのコンソールから実行できる。

Visual Studioで開いて実行する

1. 「プロジェクトやソリューションを開く」から VectorExamples.sln を開く。従来形式の `.sln` と `.csproj` はこのディレクトリに並んでいる。
2. ソリューションエクスプローラーに `VectorExamples` が1件表示されることを確認する。必要なら右クリックで「スタートアッププロジェクトに設定」を選ぶ。
3. 構成を **Debug**、プラットフォームを **Any CPU** にする。
4. **Ctrl**+**Shift**+**B** でソリューションをビルドする。
5. **Ctrl**+**F5** でデバッグなし実行、**F5** でデバッグ実行する。最後に「すべての 73 チェックに成功しました。」と表示される。

F9／F11で数式を追う

1. Program.cs の `DotLengthAndAngle` にある `double dot = VectorMath.Dot2(a, b);` の行に **F9** でブレークポイントを置く。
2. **F5** で停止し、「ローカル」や「ウォッチ」で `a.X=3`、`a.Y=4`、`b.X=2`、`b.Y=0` を確認する。
3. **F11** で VectorMath.cs の `Dot2` へ入り、`a.X * b.X + a.Y * b.Y` が `3*2 + 4*0` に対応することを確認する。
4. **Shift**+**F11** で戻り、**F10** で進めて `dot=6`、長さ5と2、角度を確認する。メソッド名にカーソルを合わせると、XMLコメントの入力・戻り値・前提を読める。

射影では `ProjectionAndOrthogonality` の `TryProject2` 呼び出し行へブレークポイントを移す。F11で入り、`numerator=6`、`denominator=4`、`factor=1.5`、射影 `(3,0)` を追う。射影先の長さを補正する分母に注目する。

法線では `TriangleNormal3D` の `TryTriangleNormal3` 呼び出しから入り、2辺・外積・長さ6・長さによる除算を追う。後の一直線の呼び出しでは、長さ0で `false` を返し、正規化へ進まないことを確認できる。角度のクランプは `DegenerateAndPrecision` の同方向の呼び出しから `TryAngle2` に入り、`cosine` と `Math.Clamp` の行を見る。

コンソールがすぐ閉じる場合

出力を読むには **Ctrl+F5** を使う。F5で終了前に止める場合は、`Main` の成功メッセージを出す `Console.WriteLine` にブレークポイントを置く。CLIから実行すれば起動元の端末に出力が残る。待機用の `ReadKey` は不要。

CLIで実行する

リポジトリのルートから、次のように実行する。

```
dotnet build sessions/02-vector/examples/VectorExamples.sln --configuration Debug
dotnet build sessions/02-vector/examples/VectorExamples.sln --configuration Release
dotnet run --project sessions/02-vector/examples/VectorExamples.csproj --configuration
Release --no-build
```

Debugで実行する場合は、上のrunコマンドの `Release` を `Debug` にする。この `examples/` を作業ディレクトリにした場合は、次のコマンドでビルドと実行ができる。

```
dotnet run
```

チェックが全件成功すると終了コード0。不一致・非有限値・期待しない成否は `FAIL` と該当ラベルを出し、終了コード1で止まる。数値の失敗では成分番号・実値・期待値・許容誤差も出す。`bin/`・`obj/`・`.vs/`・`*.csproj.user` は既存の `.gitignore` で管理対象外になる。

ファイルと読む順序

ファイル	内容
VectorExamples.sln	Visual Studioで開く入口。第2回専用のプロジェクト1件
VectorExamples.csproj	.NET 10のコンソール設定。追加パッケージなし
Program.cs	Mainからの実行順、入力、.NET API、期待値、数値・成否のチェック
VectorMath.cs	内積・長さ・外積の成分式と、角度・射影・単位法線のTryメソッド

`Program.Main` → 各題材の冒頭コメント → 呼び出す `VectorMath` の式 → 元の題材のAPI・期待値の順に読む。処理は固定サイズの2D／3Dに絞り、第1回のプロジェクトやコードには依存しない。

実行順・入力・期待結果

同じ直交座標系の標準基底を使い、2DはX右・Y上、3Dは右手系とする。位置ではなく方向を比べるときは、同じ基準点を引いて辺を作る。

題材・メソッド	教材との対応	主な入力と期待値	チェック数
1. DotLengthAndAngle	解説②・基本問題1	a=(3,4), b=(2,0)：内積6、長さ5/2、cos=3/5、約53.130102°	9
2. ProjectionAndOrthogonality	解説③・基本問題1/2	同じa/b：射影(3,0)、残り(0,4)、残りとbの内積0。(3,4)と(4,−3)も直交	7
3. CrossProduct3D	解説④・発展の任意補足	(2,0,0)×(0,3,0)=(0,0,6)、逆順(0,0,−6)、平行四辺形面積6・三角形面積3。一般の外積(−3,6,−3)も両入力と直交	11
4. OrientationAndArea2D	解説⑤・基本問題3	A=(1,1), B=(4,1), C=(3,3)：+6、面積3。C′ =(3,−1)は−6、C″ =(2,1)は0	10
5. TriangleNormal3D	解説⑥・発展問題	上のABCをz=0に埋め込む：外積(0,0,6)、単位法線+Z、逆順−Z。一直線・重複点では未定義	12
6. DegenerateAndPrecision	解説③/⑦	ゼロ角度・射影先を拒否、平行・ほぼ垂直、丸め、クランプ、NaN/Infinity、桁あふれ	24

一般の外積の入力は `a=(1,2,3)`、`b=(4,5,6)`。2Dの左右は外積のZ成分に相当する**スカラー**で判定し、通常の面積では絶対値を取る。3Dの外積・法線はベクトルとして区別する。

出力例（抜粋）：

```
1. 内積・長さ・角度（解説②／基本問題1）
  [OK] a・b: (6)
  [OK] 長さ |a|,|b|: (5, 2)
  [OK] cosθ: (0.6)
...
  角度（表示用・小数6桁）: 53.130102°
...
  [OK] 正射影: (3, 0)
  [OK] 残り a-q: (0, 4)
...
  [OK] u×v: (0, 0, 6)
  [OK] 一般のa×b: (-3, 6, -3)
...
  [OK] Cのスカラー外積: (6)
  [OK] C'のスカラー外積: (-6)
...
  [OK] ABCの単位法線: (0, 0, 1)
  [OK] ACBの単位法線: (0, 0, -1)
...
すべての 73 チェックに成功しました。
```

数式と実装を見る観点

- `Dot2`・`Dot3` は対応する成分の積を足す。非ゼロ長さで割るとcosになり、`TryAngle2` はその比を `Math.Clamp` してから `Math.Acos` に渡す。Vector2.Dotの公式資料 も参照できる。
- `TryProject2` は `(a・b)/(b・b)` を `b` に掛ける。残りの内積0を検証することで、単に係数を暗記せず垂直条件と結び付ける。
- `Cross3` は右手系の基底の外積から得た3成分の差を直接書く。Vector3.Cross と比較し、`Vector3.Dot` で直交も確認する。順序の反転で符号が変わる。
- `Cross2` は自作の `u.X*v.Y - u.Y*v.X` で、戻り値は `double`。このコードは `Vector2.Cross` を使わず、Z成分0の `Vector3.Cross` と比較する。Y下向きに座標を表し直すと符号の解釈が反転する。
- `TryTriangleNormal3` は頂点から2辺と外積を作り、長さで割る前に退化を検査する。既存の法線を非一様スケール等で変換する話とは区別し、逆転置による一般の法線変換は扱わない。

未定義・精度・許容誤差

`TryAngle2`・`TryProject2`・`TryTriangleNormal3` は、成功時だけ `out` の値を使う。`false` は未定義・下限以下・非有限値など、この教材の式で扱えない場合を示す。失敗時の `out` の既定値を、角度0や単位法線として使わない。ゼロを射影元にすることと、ゼロを射影先にすることも区別する。

生の `Dot`・`Cross`・`Length` は成分式そのもので、入力の有限性や桁あふれを補正しない。`Try` メソッドと自動チェックで有限性を検査する。角度のクランプは有限なcosの小さなはみ出しへの対処で、ゼロ長さやNaNの代わりにはならない。

条件	教材で使う値と尺度
doubleの数値比較	絶対誤差1e-12
floatを含む数値比較	絶対誤差1e-5
角度・射影先の長さの下限	長さ1e-12。単位は座標と同じ
法線を作る外積の長さの下限	1e-12。単位は長さの2乗で、三角形面積の2倍に当たる
ほぼ平行／ほぼ垂直	外積の長さ／長さの積、または内積の絶対値／長さの積を1e-12と比較。無次元

これらは小さな固定入力に対する例であり、長さと面積と角度に同じ意味のしきい値を使っているわけではない。単位やスケールを変える場合は条件を見直す。極端な値へのスケーリングや厳密な位相判定は対象外とし、ほぼ一直線でも下限以下なら法線を返さない。

題材6では `0.1+0.2` を含むベクトルの内積に微小な残差が出ることと、`1+1e-13` の差がfloatにすると消えることを示す。floatに丸めて失われた差は、後からdoubleにしても戻らない。これらの小数例は誤差を見るための補足で、講義・宿題の共有例は維持している。

比較は未丸めの値で行う。通常の数値表示は不変カルチャーの有効数字9桁（G9）、角度の度表示は小数6桁（F6）、微小残差はG17で表示する。表示文字列や丸めた期待値では判定しない。残差の末尾は実行環境で異なる場合がある。

自動チェックとCI

自動チェックは実行ごとに73件行う。`[OK]` は数値比較と条件確認をそれぞれ1件として数える。Check Vector Examples が第2回のC#コード・プロジェクト・ソリューション・ビルド設定・ワークフロー自身の変更を、PR時とmainへのpush時に検証する。Markdownだけの変更では起動しない。

.NET 10 SDKで `.sln` を復元・Releaseビルドし、このプロジェクトを実行する。終了コード0、実際の `[OK]` が73件、最後の「すべての 73 チェックに成功しました。」が揃っていれば成功する。チェック件数を変更するときは内容をレビューし、ワークフローの `EXPECTED_CHECKS` と、このREADMEの件数・出力例を同じPRで更新する。

PRのChecks、または **Actions → Check Vector Examples → 対象の実行 → VectorExamples (Release)** で確認できる。失敗時は `Restore solution`・`Build solution (Release)`・`Run VectorExamples and verify all 73 checks` の該当ログを開く。`FAIL` があればラベル・成分番号・実値・期待値・許容誤差、件数や成功要約の不一致なら `::error::` のメッセージを確認する。

このCIは `contents: read` の読み取り専用で、Secretや書き込み権限を要求しない。第1回の43チェックとCI、PDFの生成・自動コミットとは独立している。Visual StudioのF5／Ctrl+F5／F9／F11はCIでは検証できないため、上記の手順でローカル確認する。

資料の元データ（GitHubリポジトリ）： https://github.com/mkunori/geometry-study-group